

REST

20 July 2026 02:50 PM

REST (Representational State Transfer) is an architectural style for designing APIs. It treats everything as a **resource**, where each resource is identified by a unique URL. Clients interact with these resources using standard HTTP methods.

Example

```
GET /users/123 → Get a user
POST /users → Create a user
PUT /users/123 → Update a user
DELETE /users/123 → Delete a user
```

Advantages

- Easy to learn and widely used.
- Uses standard HTTP methods.
- Supports browser and CDN caching.
- Works well for CRUD applications.
- Mature ecosystem with excellent tooling.

Disadvantages

- **Over-fetching:** Server may send more data than the client needs.
- **Under-fetching:** Client may need multiple API calls to gather related data.
- API versioning (/v1, /v2) is often required when responses change.

Best Use Cases

- E-commerce APIs
- Banking APIs
- CRUD applications
- Public APIs
- Third-party integrations

GraphQL

What is GraphQL?

GraphQL is a **query language for APIs** developed by **Facebook (Meta)** in **2015**. Unlike REST, where the **server decides** what data to return, in GraphQL the **client decides exactly what data it needs**.

Instead of having multiple endpoints (/users, /posts, /orders), GraphQL exposes a **single endpoint** (usually /graphql) where clients send queries to fetch or modify data.

Why was GraphQL introduced?

REST works well for simple applications, but as applications became more complex, it had some limitations:

- **Over-fetching:** Server returns unnecessary data.
- **Under-fetching:** Client needs multiple API calls to get related data.
- Different clients (Web, Mobile, IoT) often need different fields.

GraphQL solves these problems by allowing the client to request only the required data.

How GraphQL Works

Unlike REST, which has many endpoints, GraphQL usually has **one endpoint**.

```
Client
|
POST /graphql
|
GraphQL Server
|
Database / Microservices
The client sends a query describing exactly what it wants.
Example:
```

```
query {
  user(id: 123) {
    name
    email
    posts {
      title
    }
  }
}
```

Response:

```
{
  "data": {
    "user": {
      "name": "John",
      "email": "john@gmail.com",
      "posts": [
```

```

    {
      "title": "GraphQL Basics"
    }
  ]
}
}
}

```

Notice that only **name**, **email**, and **post title** are returned—nothing else.

Core Components of GraphQL

1. Query (Read Data)

A **Query** is used to retrieve data from the server.

Equivalent REST method:

GET

Example:

```

query {
  product(id: 1) {
    name
    price
  }
}

```

Returns only the requested fields.

2. Mutation (Modify Data)

Mutations are used to:

- Create
- Update
- Delete

Equivalent REST methods:

POST

PUT

PATCH

DELETE

Example:

```

mutation {
  createUser(name:"Rahul") {
    id
    name
  }
}

```

3. Subscription (Real-Time Updates)

Subscriptions allow clients to receive updates automatically whenever data changes.

Useful for:

- Chat Applications
- Live Sports Scores
- Stock Market
- Notifications

Example:

```

subscription {
  newMessage {
    sender
    text
  }
}

```

Whenever a new message arrives, the client receives it instantly.

GraphQL Schema

A **Schema** defines:

- Available data
- Data types
- Queries
- Mutations
- Relationships

Example:

```

type User {
  id: ID!
  name: String!
  email: String!
}

```

Think of the schema as the **contract** between the client and server.

Advantages of GraphQL

☒ 1. No Over-Fetching

Client requests only required fields.

REST

```
{
  "name": "John",
  "email": "john@gmail.com",
  "address": "London",
  "phone": "999999999"
}
```

Need only name?

Still receives everything.

GraphQL

```
query{
  user{
    name
  }
}
```

Returns only

```
{
  "name": "John"
}
```

☒ 2. No Under-Fetching

REST

GET /users/1

GET /users/1/posts

GET /posts/10/comments

Three requests.

GraphQL

```
query{
  user(id:1){
    name
    posts{
      title
      comments{
        text
      }
    }
  }
}
```

One request.

☒ 3. Single Endpoint

REST

/users

/orders

/products

/payments

GraphQL

/graphql

Much simpler for the client.

☒ 4. Strongly Typed

Every field has a type.

Example

email: String

age: Int

This improves documentation and reduces errors.

☒ 5. No Versioning

REST often needs

/v1/users

/v2/users

GraphQL simply adds new fields without breaking existing clients.

☑ 6. Better for Multiple Clients

Different devices require different data.

Example

Web App needs

- Name
- Email
- Address
- Orders

Mobile App needs

- Name
- Profile Picture

GraphQL allows each client to request exactly what it needs.

Disadvantages of GraphQL

✗ More Complex

Requires:

- GraphQL Server
- Schema
- Resolvers

Setup is more complicated than REST.

✗ Difficult Caching

REST uses HTTP GET caching easily.

GraphQL mostly uses POST requests, making caching more challenging.

✗ Heavy Queries

A client could request deeply nested data.

Example:

User

↓

Orders

↓

Products

↓

Reviews

↓

Comments

This can generate expensive database queries and impact performance.

✗ Higher Server Load

Because clients control the query, the server must carefully optimize execution to prevent excessive work.

✗ Security Risks

Without safeguards, complex queries can consume significant resources.

To mitigate this:

- Query depth limits
- Rate limiting
- Query cost analysis
- Authentication & authorization

are commonly applied.

When to Use GraphQL

Choose GraphQL when:

- Multiple clients need different data.
- Applications have deeply connected data.
- Mobile apps need to minimize network usage.
- Real-time updates are important.
- Flexible querying is required.

Examples:

- Facebook
- GitHub API
- Shopify
- Dashboard applications

When NOT to Use GraphQL

Prefer REST when:

- Building simple CRUD applications.
- HTTP caching is important.

- Third-party integrations expect REST.
- Your team wants a simpler implementation.